

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ  
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

**Дисциплина:** Бэк-энд разработка

Отчет

Лабораторная работа №1

«Реализация REST API на основе boilerplate»

Выполнил:

Якшин Артемий

Группа БР1.1

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

## Задача

Используя подходящий boilerplate (в качестве основы взят стек *Express + TypeORM*), с учётом результатов ДЗ1 (проектирование базы данных) и ДЗ2 (технический дизайн API в формате OpenAPI), реализовать REST API для выбранного варианта — **приложение для бронирования столиков в ресторанах**. В рамках работы необходимо реализовать:

- модели (сущности базы данных);
- представления (формирование тел запросов и ответов);
- контроллеры (обработка маршрутов и бизнес-логика).

Результатом является полностью работающее API, соответствующее спроектированной OpenAPI-схеме и диаграмме базы данных.

## Ход работы

### 1. Выбор технологического стека

В качестве основы выбран стек, рекомендованный в курсе:

| Компонент        | Технология             | Назначение                                           |
|------------------|------------------------|------------------------------------------------------|
| Среда выполнения | Node.js + TypeScript   | Серверная платформа, статическая типизация           |
| Web-фреймворк    | Express 4              | Маршрутизация, middleware                            |
| ORM              | TypeORM 0.3            | Модели, миграции (synchronize), запросы              |
| СУБД             | SQLite                 | Хранение данных (файл <code>database.sqlite</code> ) |
| Аутентификация   | jsonwebtoken, bcryptjs | JWT-токены, хеширование паролей                      |
| Прочее           | dotenv, cors           | Конфигурация окружения, CORS                         |

### 2. Структура проекта

Проект организован по слоям: `entities` (модели), `controllers` (представления и бизнес-логика), `routes` (контроллеры маршрутов), `middleware` (авторизация, обработка ошибок), `utils` (вспомогательные модули).

```

src/
├── config/data-source.ts           # инициализация TypeORM + SQLite
├── entities/                      # модели (TypeORM @Entity)
│   ├── User.ts                   ├── Restaurant.ts           ├── Cuisine.ts
│   ├── RestaurantPhoto.ts        ├── MenuItem.ts          ├── RestaurantTable.ts
│   ├── Reservation.ts            └── Review.ts
├── controllers/                  # бизнес-логика (представления)
│   ├── auth.controller.ts         ├── users.controller.ts
│   ├── restaurants.controller.ts  ├── cuisines.controller.ts
│   ├── tables.controller.ts       ├── reservations.controller.ts
│   └── reviews.controller.ts
├── routes/                       # маршруты Express (контроллеры)
│   ├── index.ts + auth/users/restaurants/cuisines/reservations/reviews
├── middleware/
│   ├── auth.ts                   # JWT-авторизация
│   └── error-handler.ts          # централизованная обработка ошибок
├── utils/
│   ├── jwt.ts                   ├── errors.ts               ├── async-handler.ts
│   ├── serializers.ts           # формирование тел ответов по OpenAPI
│   └── restaurant-stats.ts      # агрегация рейтингов и фото
├── index.ts                      # точка входа Express-приложения
└── seed.ts                      # наполнение БД тестовыми данными

```

### 3. Модели (сущности)

На основе ER-диаграммы из ДЗ1 реализовано 8 сущностей TypeORM. Связи:

User 1-N Reservation, User 1-N Review, Restaurant 1-N RestaurantTable / MenuItem / RestaurantPhoto / Review, RestaurantTable 1-N Reservation, Restaurant N-M Cuisine (через таблицу связи restaurant\_cuisines). На таблице reviews задано ограничение уникальности пары (user\_id, restaurant\_id) — один отзыв пользователя на ресторан.

| Сущность        | Таблица           | Ключевые поля                                                                                               |
|-----------------|-------------------|-------------------------------------------------------------------------------------------------------------|
| User            | users             | user_id, name, email (unique), phone, password_hash, created_at                                             |
| Restaurant      | restaurants       | restaurant_id, name, description, address, city, price_level, opening_time, closing_time, phone, created_at |
| Cuisine         | cuisines          | cuisine_id, name (unique)                                                                                   |
| RestaurantPhoto | restaurant_photos | photo_id, restaurant_id, photo_url, is_main                                                                 |
| MenuItem        | menu_items        | menu_item_id, restaurant_id, name, description, price, category                                             |
| RestaurantTable | restaurant_tables | table_id, restaurant_id, table_number, capacity, status                                                     |

|             |              |                                                                                                         |
|-------------|--------------|---------------------------------------------------------------------------------------------------------|
| Reservation | reservations | reservation_id, user_id, table_id, reservation_date, reservation_time, guests_count, status, created_at |
| Review      | reviews      | review_id, user_id, restaurant_id, rating, comment, created_at                                          |

Пример объявления модели (src/entities/Reservation.ts):

```
@Entity('reservations')
export class Reservation {
  @PrimaryGeneratedColumn({ name: 'reservation_id' })
  reservation_id!: number;

  @Column({ type: 'integer' }) user_id!: number;
  @Column({ type: 'integer' }) table_id!: number;
  @Column({ type: 'date' }) reservation_date!: string;
  @Column({ type: 'varchar', length: 5 }) reservation_time!: string;
  @Column({ type: 'integer' }) guests_count!: number;
  @Column({ type: 'varchar', length: 20, default: 'pending' }) status!:
ReservationStatus;
  @CreateDateColumn({ name: 'created_at' }) created_at!: Date;

  @ManyToOne(() => User, (u) => u.reservations, { onDelete: 'CASCADE' })
  @JoinColumn({ name: 'user_id' }) user!: User;

  @ManyToOne(() => RestaurantTable, (t) => t.reservations, { onDelete:
'CASCADE' })
  @JoinColumn({ name: 'table_id' }) table!: RestaurantTable;
}
```

4. Контроллеры и маршруты

Реализовано 7 контроллеров. Маршруты собраны под префиксом /api/v1. Маршруты, помеченные  , требуют заголовок Authorization: Bearer <token>.

| Метод и путь                                                                                                   | Описание                                                                      |
|----------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------|
| POST /auth/register                                                                                            | Регистрация пользователя → { token, user }                                    |
| POST /auth/login                                                                                               | Вход → { token, user }                                                        |
|  GET /users/me              | Профиль текущего пользователя                                                 |
|  PUT /users/me              | Обновление профиля (name, phone, password)                                    |
|  GET /users/me/reservations | История бронирований (фильтр по status, пагинация)                            |
| GET /restaurants                                                                                               | Список ресторанов (фильтры: city, cuisine_id, price_level, search; пагинация) |

|                                                                                                             |                                                        |
|-------------------------------------------------------------------------------------------------------------|--------------------------------------------------------|
| GET /restaurants/{id}                                                                                       | Детали ресторана (с меню и фото)                       |
| GET /restaurants/{id}/photos                                                                                | Фотографии ресторана                                   |
| GET /restaurants/{id}/menu                                                                                  | Меню ресторана (фильтр по category)                    |
| GET /restaurants/{id}/reviews                                                                               | Отзывы о ресторане (пагинация, средний рейтинг)        |
| GET /restaurants/{id}/tables                                                                                | Доступные столики на дату/время (учёт активных броней) |
| GET /cuisines                                                                                               | Все типы кухонь                                        |
|  POST /reservations        | Создание бронирования                                  |
|  GET /reservations/{id}    | Детали бронирования (только владелец)                  |
|  PUT /reservations/{id}    | Изменение бронирования                                 |
|  DELETE /reservations/{id} | Отмена бронирования (статус cancelled)                 |
|  POST /reviews             | Создание отзыва (один на ресторан от пользователя)     |
|  PUT /reviews/{id}       | Редактирование отзыва (только автор)                   |
|  DELETE /reviews/{id}    | Удаление отзыва (только автор)                         |

Пример контроллера (создание бронирования, фрагмент `reservations.controller.ts`):

```
export const create = async (req: Request, res: Response) => {
  const { table_id, reservation_date, reservation_time, guests_count } =
    req.body;
  // ... валидация полей -> 400 Bad Request с details ...

  const table = await tableRepo.findOne({ where: { table_id } });
  if (!table) throw NotFound('Table not found');
  if (guests_count > table.capacity) throw BadRequest(`Table capacity is
    ${table.capacity}`);

  if (await isTableTaken(table_id, reservation_date, reservation_time))
    throw Conflict('Table is already booked for this date and time');

  const r = repo.create({ user_id: req.user!.user_id, table_id,
    reservation_date, reservation_time, guests_count, status: 'confirmed' });
  await repo.save(r);
  res.status(201).json(serializeReservation(await
    loadReservation(r.reservation_id)));
};
```

## 5. Представления, аутентификация и обработка ошибок

Тела ответов формируются модулем `utils/serializers.ts` в строгом соответствии с OpenAPI-схемой (например, у пользователя не возвращается `password_hash`). Аутентификация — JWT: при регистрации/входе выдаётся токен, middleware `authMiddleware` извлекает пользователя из заголовка `Authorization`. Пароли хешируются `bcrypt` (10 раундов).

Ошибки обрабатываются централизованно. Используются типизированные исключения `HttpError`, единый формат ответа — `{ error, message, details? }`. Реализованы коды `400` (валидация), `401` (нет/неверный токен), `403` (нет прав), `404` (не найдено), `409` (конфликт), `500`.

## 6. Тестовые данные

Скрипт `npm run seed` наполняет БД: 2 пользователя, 5 типов кухонь, 4 ресторана с фотографиями, меню, столиками и отзывами. Тестовые учётные записи: `ivan@example.com / securepass123`, `maria@example.com / mariapass456`.

## 7. Запуск

```
cd "BP1.1/Якшин Артемий/labs/lab1"
cp .env.example .env
npm install
npm run seed          # наполнение БД тестовыми данными
npm run dev           # dev-сервер: http://localhost:3000/api/v1
```

## 8. Проверка работоспособности

API проверено сквозным сценарием через `curl`. Ниже — характерные запросы и ответы.

**Регистрация и получение профиля:**

```
$ curl -s -X POST http://localhost:3000/api/v1/auth/register \
  -H 'Content-Type: application/json' \
  -d '{"name":"Test User","email":"test1@example.com","phone":"+79991112233","password":"testpass123"}'
{"token":"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...","user":
{"user_id":3,"name":"Test User",
  "email":"test1@example.com","phone":"+79991112233","created_at":"2026-05-09T17:06:38.000Z"}}

$ curl -s http://localhost:3000/api/v1/users/me -H "Authorization: Bearer $TOKEN"
{"user_id":3,"name":"Test User","email":"test1@example.com","phone":"+79991112233",
  "created_at":"2026-05-09T17:06:38.000Z"}

$ curl -s http://localhost:3000/api/v1/users/me # без токена
HTTP 401 {"error":"Unauthorized","message":"Missing or invalid authorization token"}
```

## Ошибка валидации (400) и конфликт email (409):

```
$ curl -s -X POST .../auth/register -d
'{"name":"x","email":"bad","password":"short"}'
HTTP 400 {"error":"Bad Request","message":"Validation failed","details":[
  {"field":"name","message":"Must be 2-100 characters"},
  {"field":"email","message":"Must be a valid email address"},
  {"field":"password","message":"Must be 8-64 characters"}]}

$ curl -s -X POST .../auth/register -d '{"name":"Test User","email":"test1@example.com","password":"testpass123"}'
HTTP 409 {"error":"Conflict","message":"User with this email already exists"}
```

## Список ресторанов с фильтром и детальная карточка:

```
$ curl -s --get --data-urlencode "city=Москва"
http://localhost:3000/api/v1/restaurants
{"data":[{"restaurant_id":1,"name":"La Piazza","address":"ул. Тверская, 10, Москва",
  "city":"Москва","price_level":"$$","cuisines":
[{"cuisine_id":1,"name":"Итальянская"},
 {"cuisine_id":5,"name":"Французская"}],"average_rating":4.5,
  "main_photo":"https://example.com/photos/lapiazza-1.jpg"},
...],"total":3,"page":1,"limit":20}

$ curl -s http://localhost:3000/api/v1/restaurants/1
{"restaurant_id":1,"name":"La Piazza", ... ,"average_rating":4.5,
  "photos":[{"photo_id":1,"photo_url":"https://example.com/photos/lapiazza-1.jpg"}, ...],
  "menu_items":[{"menu_item_id":1,"name":"Паста Карбонара","price":650,"category":"Основные блюда"}, ...],
  "reviews_count":2}
```

## Бронирование столика и проверка занятости:

```
$ curl -s --get --data-urlencode "date=2026-04-15" --data-urlencode
"time=19:00" \
    http://localhost:3000/api/v1/restaurants/1/tables
[{"table_id":1,"table_number":1,"capacity":2,"status":"available"}, ...]

$ curl -s -X POST http://localhost:3000/api/v1/reservations -H
"Authorization: Bearer $TOKEN" \
    -d '{"table_id":1,"reservation_date":"2026-04-
15","reservation_time":"19:00","guests_count":2}'
{"reservation_id":1,"user_id":1,"table":
{"table_id":1,"table_number":1,"capacity":2},
"restaurant":{"restaurant_id":1,"name":"La Piazza",
...},"reservation_date":"2026-04-15",

"reservation_time":"19:00","guests_count":2,"status":"confirmed","created_at"
:"..."}

$ curl -s -X POST .../reservations -d
'{"table_id":1,"reservation_date":"2026-04-
15","reservation_time":"19:00","guests_count":2}'
HTTP 409 {"error":"Conflict","message":"Table is already booked for this
date and time"}

$ curl -s --get --data-urlencode "date=2026-04-15" --data-urlencode
"time=19:00" .../restaurants/1/tables
[{"table_id":1,"table_number":1,"capacity":2,"status":"reserved"}, # столик
теперь занят
{"table_id":2,"table_number":2,"capacity":2,"status":"available"}, ...]
```

## Отзывы (создание, ограничение «один на ресторан», список):

```
$ curl -s -X POST http://localhost:3000/api/v1/reviews -H "Authorization:
Bearer $TOKEN" \
    -d '{"restaurant_id":2,"rating":5,"comment":"Отличный японский
ресторан"}'
{"review_id":4,"user":{"user_id":1,"name":"Иван Иванов",
...},"restaurant_id":2,
"rating":5,"comment":"Отличный японский ресторан","created_at":"..."}

$ curl -s -X POST .../reviews -d '{"restaurant_id":1,"rating":4}' #
повторный отзыв на тот же ресторан
HTTP 409 {"error":"Conflict","message":"You have already reviewed this
restaurant"}

$ curl -s http://localhost:3000/api/v1/restaurants/1/reviews
{"data":[{"review_id":1,"user":{"name":"Иван Иванов",
...},"rating":5,"comment":"Отличный ресторан, вкусная еда!"},
{"review_id":2,"user":{"name":"Мария Петрова",
...},"rating":4,"comment":"Хорошее место, но чуть шумно."}],
"total":2,"average_rating":4.5,"page":1,"limit":10}
```

Также проверены: `PUT /reservations/{id}` (перенос времени), `DELETE /reservations/{id}` (статус `→ cancelled`, ответ `204`), `GET /users/me/reservations` (история), `GET /restaurants/{id}/menu`, `GET /cuisines`, ответ `404` на несуществующий ресурс, ответ `403` при доступе к чужому бронированию. Проверка типов `tsc --noEmit` проходит без ошибок.

## Вывод

В ходе лабораторной работы на основе стека *Node.js + TypeScript + Express + TypeORM + SQLite* реализовано полностью работающее REST API приложения для бронирования столиков в ресторанах. Спроектированы и реализованы 8 моделей данных (с учётом ER-диаграммы из Д31), 7 контроллеров и все эндпоинты согласно OpenAPI-схеме из Д32, включая аутентификацию по JWT, хеширование паролей, фильтрацию и пагинацию списков, проверку доступности столиков и централизованную обработку ошибок с корректными HTTP-статусами. Работоспособность API подтверждена сквозным сценарием тестирования (регистрация `→` вход `→` поиск ресторанов `→` выбор столика `→` бронирование `→` история `→` отзыв).